NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

GROUP ASSIGNMENT REPORT

Midterm

Web Design and Programming ([TO BE CONFIRMED])

EventHub: An Event Ticketing Platform

Traveloka-style discovery, booking, and role-based operations

Group: Group EventHub
Class: DS66B
Instructor: M.Sc. Le Duy Khanh
Semester: Semester 1, 2026–2027
GitHub: https://github.com/baokhanh1410/web_design

Team Members

No.	Full name	Student ID	Role
1	Nguyễn Bảo Khánh	11247178	Team lead, back-end
2	Chu Đức Duy	11247159	Front-end, documentation
3	Trần Quang Huy	11247176	Database, organizer module

Hanoi, November 2026

Declaration

We declare that this report is the result of our group's own work for the course *Web Design and Programming*. All sources (literature, source code, images, AI tools) are cited explicitly. We take full responsibility for the submitted content.

Task Allocation and Contributions

Member	Work done	Contribution (%)
Nguyễn Bảo Khánh	API design, authentication, repository coordination	34
Chu Đức Duy	Interface implementation, check-in flows, report writing	33
Trần Quang Huy	Schema design, SQL views, organizer-facing features	33
	Total	**100**

The split reflects the roles recorded on the cover page as of 11 November 2026.

Table of Contents

1 Introduction . . . 5
1.1 Background and motivation . . . 5
1.2 Objectives . . . 5
1.3 Scope and target users . . . 5
1.4 Technologies . . . 5
1.5 Report structure . . . 6
2 Analysis and Design . . . 6
2.1 Requirements . . . 6
2.1.1 Functional requirements . . . 6
2.1.2 Non-functional requirements . . . 6
2.2 Interface design (UI/UX) . . . 7
2.3 System architecture . . . 7
2.4 Database design . . . 9
2.5 API design . . . 10
3 Implementation . . . 10
3.1 Project structure . . . 10
3.2 User interface . . . 11
3.3 Server-side logic . . . 11
3.4 Source control and teamwork . . . 17
3.5 Deployment . . . 17
4 Results and Conclusion . . . 17
4.1 Testing . . . 17
4.2 Achievements and limitations . . . 18
4.3 Lessons learned and future work . . . 18
References . . . 20
A Appendix . . . 20
A.1 Setup and run instructions . . . 20
A.2 Demo accounts . . . 21
A.3 Use of AI tools . . . 21

List of Figures

1 **Landing hero.** Search widget, trust badges, and category shortcuts on `landing.html`. The header shows the Admin chip because the default stored role is Admin. 7

2 **Catalogue with Admin menu.** Explore view on `index.html`: hero, filters, category pills, and a dropdown with role switcher plus admin links. 8

3 **Staff catalogue.** The same explore view without "Tạo sự kiện" or "Quản trị & ML" in the header. The user chip is Chu Đức Duy / Staff. 8

4 **Guest catalogue.** Header reduced to Trang chủ, Khám phá, and Vé của tôi. Role switcher remains for demonstration. 9

5 **Runtime architecture.** The UI always runs. When Uvicorn is absent, the catalogue falls back to `INITIAL_EVENTS` in `app.js`. Screenshots in this report used that independent mode. 9

6 **Entity–relationship diagram.** Events sit between venues and organisers; registrations connect guests to events; users bind a role and an optional guest profile. Source: `docs/ERD.png`. Table definitions are in `sql/schema.sql`. 12

7 **Catalogue filters.** Keyword, category list, date picker, and maximum price (VND) on the explore hero. Staff session. 12

8 **Three-step booking wizard.** (1) Standard ticket at 450,000 VND for Monsoon Music Festival 2026. (2) Attendee Chu Đức Duy. (3) Voucher field and payable total before discount. 13

9 **E-Ticket.** Code `EVT-75564922` for one Standard ticket, status Hợp lệ. The matrix is an SVG pattern derived from a string hash, not a standards-compliant QR encoder. 13

10 **Profile — tickets.** Staff account Chu Đức Duy. Counters show 5 purchased, 4 attended, 3 liked. Tab "Đã tham gia (4)" lists issued codes including `EVT-75564922`. 13

11 **Admin operations console.** Four KPIs (revenue, managed events, tickets, occupancy), event table with occupancy bars, and actions Xem / Hủy. Banner: Admin Mode. 14

12 **Create-event modal.** Name, category, start time, venue, price, capacity, and Unsplash banner URL. Shown to Admin and Organizer only. 14

13 **Landing promotion.** New-member 20% banner with code `WELCOME20`. The booking wizard instead recognises `TRAVELOKA20` and `WELCOME50`; the two surfaces are not the same code list. . . . 15

14 **Landing footer.** Category links, partner links, payment badges (display only), and contact `ds66b@neu.edu.vn`. 15

15 **How-to strip and marketing counters.** Three steps (search, pay, receive QR). The numeric counters were zero in the captured session. 16

List of Tables

1 **Technologies used in EventHub.** Entries match `requirements.txt` and the `web/` source tree. . 6

2 **Published REST endpoints.** Paths are taken from `api/main.py`. Check-in on the API requires Organizer or Admin (see Section 4.2). 10

3 **Selected manual test cases.** TC14–TC15 failed on the captured pages. TC16 was not executed because the session used a local file URL without Uvicorn. 17

4 **Voucher mismatch (TC14).** The landing copy and the wizard do not share a catalogue. 18

5 **Demo accounts.** Source: project README. Mock-mode role buttons do not call `/api/auth/login`. 21

1. Introduction

EventHub is a web application for discovering events, purchasing tickets, and operating venues under role-based access control. This section states the problem, the product goals, the in-scope features, and the technologies that implement them.

1.1. Background and motivation

Event tickets in Vietnam are still sold through mixed channels: social posts, spreadsheets, and a few large aggregators. Buyers need searchable catalogues, instant confirmation, and a machine-readable pass at the gate. Organisers need occupancy, revenue, and check-in in one place.

The course treats the web as a client–server system over HTTP [8, 4]. A ticketing product is a direct application of that model: HTML/CSS/JavaScript in the browser, a FastAPI service, and a relational store [7, 10]. The group therefore built EventHub as a Traveloka-style storefront with four demo roles (Admin, Organizer, Staff, Guest) rather than a generic CRUD sample.

1.2. Objectives

The project set three measurable objectives.

- Deliver a responsive catalogue with keyword, category, date, and maximum-price filters, plus a three-step booking wizard that issues an E-Ticket with a QR-style code.
- Persist events, guests, registrations, finance, and RBAC data in PostgreSQL, and expose them through a FastAPI REST layer with bcrypt password hashes and JWT access tokens [6, 3].
- Provide role-specific operations: Admin and Organizer dashboards (KPIs, event table, pricing widget, check-in); Staff check-in without create/admin navigation; Guest browse-and-book.

1.3. Scope and target users

Target users are (i) attendees who search and buy, (ii) staff who check tickets at the gate, and (iii) organisers and administrators who publish events and inspect KPIs.

In scope: landing and catalogue pages; a booking wizard with two demo vouchers (`TRAVELOKA20` and `WELCOME50`); E-Ticket display; profile tickets and a password form; a create-event modal; the admin dashboard; JWT login and register against PostgreSQL; and capacity checks on registration.

Out of scope: a production payment gateway (Visa/MoMo/ZaloPay appear only as badges); a trained XGBoost model (the pricing pane is a JavaScript heuristic); public HTTPS deployment; automated pytest coverage (no test package is present in the repository).

1.4. Technologies

Table 1 lists the stack that is actually declared in `requirements.txt` and the `web/` tree. Libraries advertised in the README but absent from the lockfile are noted in Section 4.2.

Component	Technology	Purpose
Front-end	HTML5, CSS3, JavaScript (ES6+)	Storefront, dashboard, local state
Back-end	Python, FastAPI, Uvicorn	REST API, static file mount
Security	bcrypt, python-jose (HS256 JWT)	Password hashing, access tokens
Database	PostgreSQL, SQLAlchemy, pandas	Persistence and query results
Tooling	Git, GitHub, python-dotenv	Version control, configuration

Table 1. Technologies used in EventHub. Entries match `requirements.txt` and the `web/` source tree.

1.5. Report structure

Section 2 analyses requirements, interface, architecture, schema, and API contracts. Section 3 describes the source layout, the running interface, and the server-side logic. Section 4 reports manual tests from the captured screenshots, states limitations, and lists future work.

2. Analysis and Design

Design choices follow the client–server request/response cycle and a resource-oriented HTTP API [4, 2]. This section records requirements, the storefront layout, the runtime architecture, the relational schema, and the published endpoints.

2.1. Requirements

2.1.1. Functional requirements

FR1. Guests can browse events and filter by keyword, category, date, and maximum price.

FR2. Guests can complete a three-step booking (quantity, attendee data, voucher and total) and receive an E-Ticket code.

FR3. Authenticated guests can open a profile of purchased tickets and a password-change form.

FR4. Users can register as guests and log in with username and password; demo accounts support one-click role switching.

FR5. Admin and Organizer users can open a KPI dashboard, search the event table, cancel or restore an event, and create an event from a modal.

FR6. Admin and Organizer users can run a pricing widget that maps capacity, category, and a base price to suggested tiers.

FR7. The mock UI lets Staff, Organizer, and Admin search bookings and toggle check-in. The FastAPI route `POST /api/checkin` currently allows Organizer and Admin only.

FR8. The API can list, create, update, and cancel events; register guests; create bookings; check in a registration; and return finance and attendance summaries.

2.1.2. Non-functional requirements

The interface must remain usable on a desktop viewport (the captured sessions use a wide browser window). Passwords must be stored as bcrypt hashes, not plaintext [6, 5]. Access tokens follow JWT with an expiry claim [3]. The database must reject registrations that exceed venue capacity (trigger `fn_enforce_capacity` plus an application-

level count). The front end must degrade to local mock data when the API is unreachable. The status pill then reads "Chế độ: Độc lập".

2.2. Interface design (UI/UX)

The visual language copies a travel-commerce pattern: Inter typeface, a sticky header (logo, search, navigation, user chip), a blue hero, white filter cards, and orange primary actions. Category pills sit above a three-column event grid. Admin and Organizer see extra header links ("Tạo sự kiện", "Quản trị & ML"); Staff and Guest do not (Figures 2–4).

Figure 1 shows the marketing landing page. Figure 7 shows the catalogue filters. The booking path is a modal wizard (Figure 8); the issued pass is a ticket card with a generated QR-style SVG (Figure 9). The operations console uses four KPI cards and three tabs (Figure 11).



Figure 1. Landing hero. Search widget, trust badges, and category shortcuts on `landing.html`. The header shows the Admin chip because the default stored role is Admin.

Colour and type are defined as CSS custom properties in `web/css/style.css` (primary blue, accent orange, Inter). Layout uses Flexbox and Grid [4]. Copy and prices use Vietnamese locale (`vi-VN`) and the đồng symbol.

2.3. System architecture

The product has two cooperating runtimes, shown in Figure 5. The browser loads static HTML/CSS/JS. JavaScript holds application state in `localStorage` (`eventhub_user`, `eventhub_current_role`, `eventhub_events`, `eventhub_bookings`). On load it probes `http://{host}:8000/api/events`. A successful JSON list replaces the mock catalogue and sets `apiConnected`. Login posts JSON to `/api/auth/login` and stores the bearer token when the server answers.

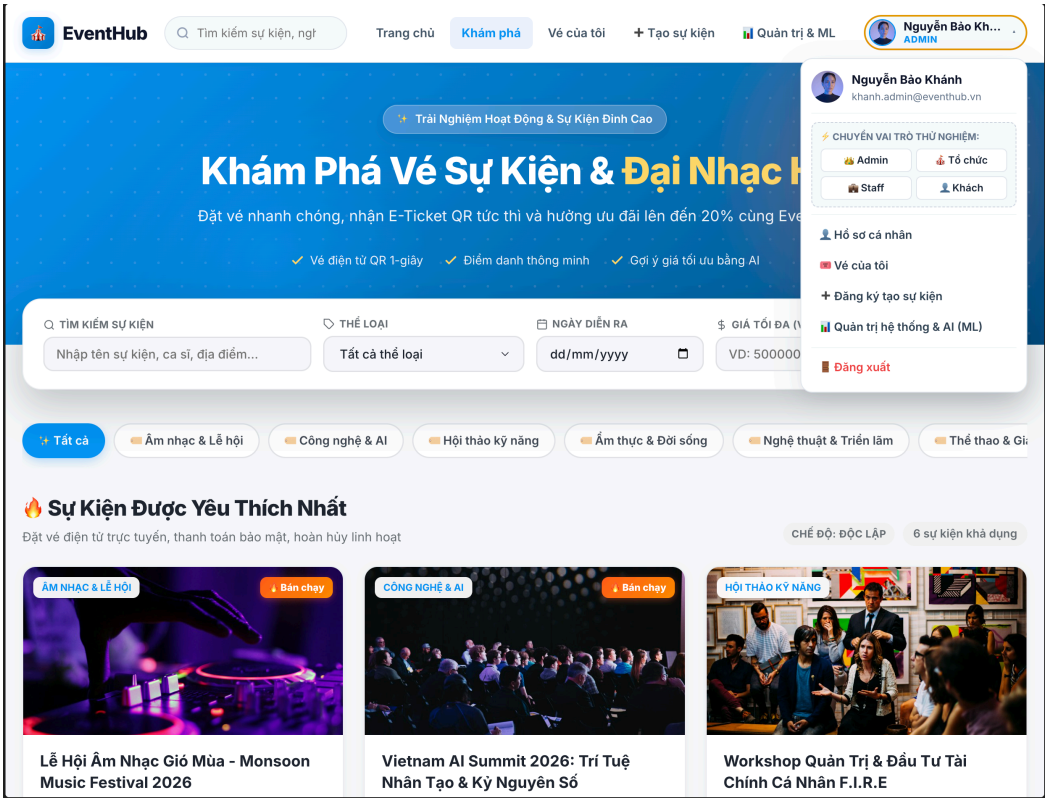


Figure 2. Catalogue with Admin menu. Explore view on `index.html`: hero, filters, category pills, and a dropdown with role switcher plus admin links.

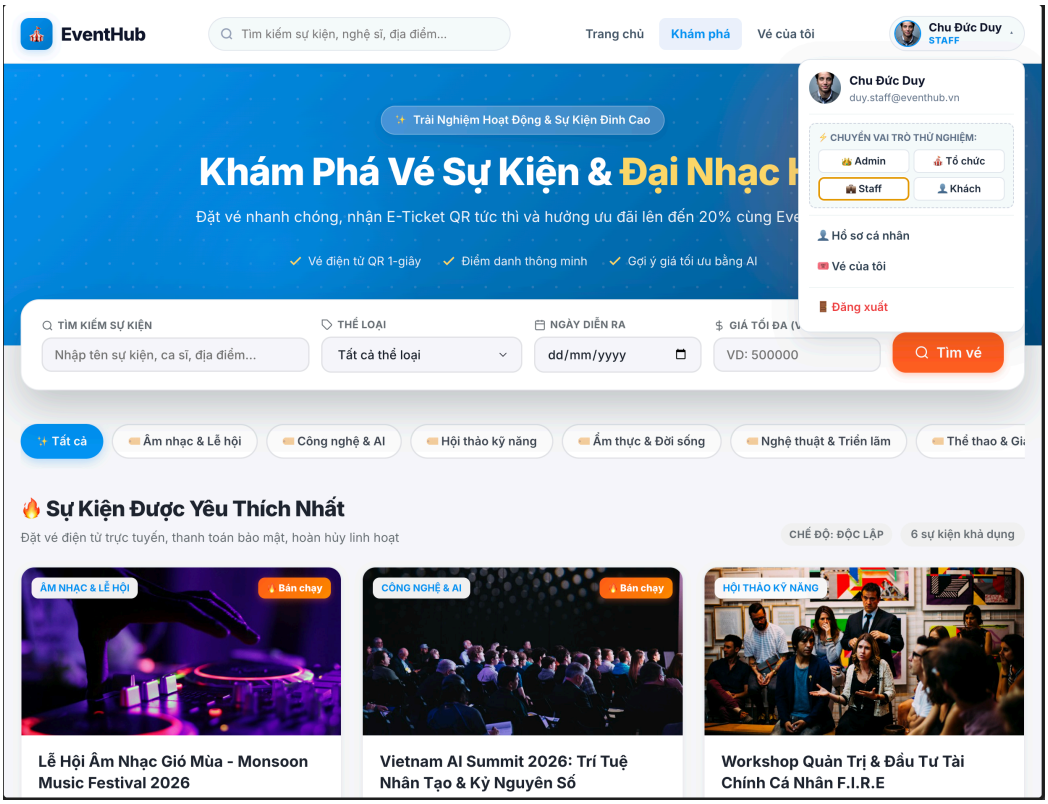


Figure 3. Staff catalogue. The same explore view without “Tạo sự kiện” or “Quản trị & ML” in the header. The user chip is Chu Đức Duy / Staff.

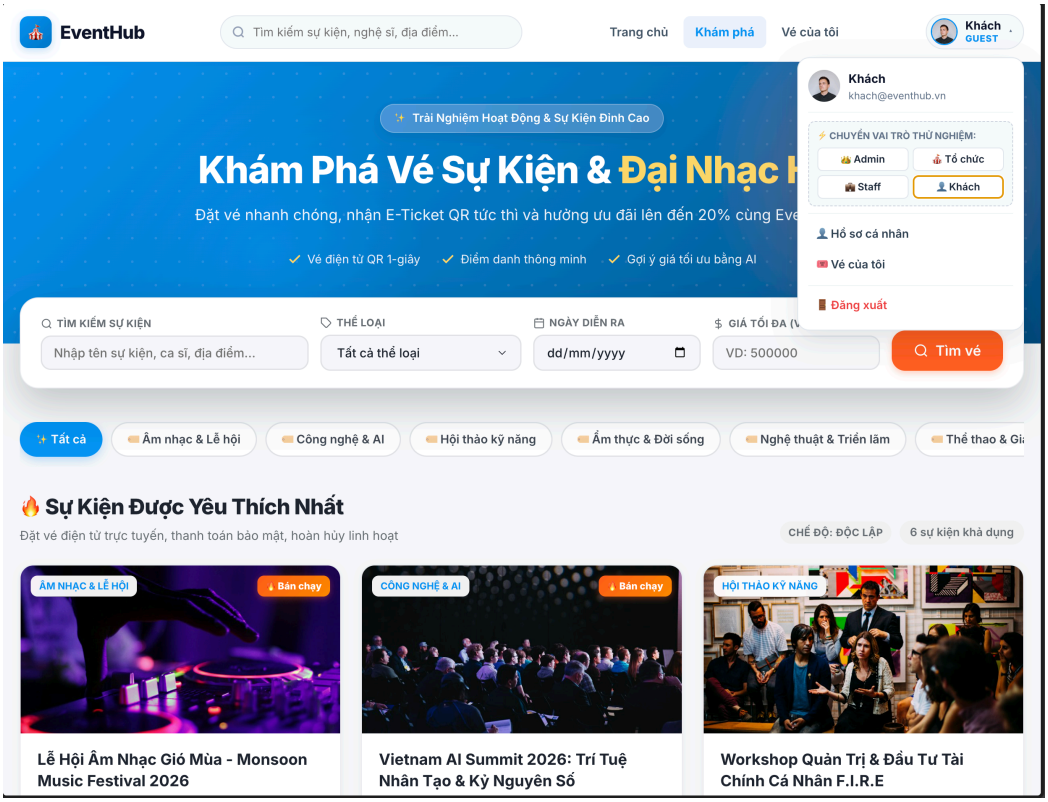


Figure 4. Guest catalogue. Header reduced to Trang chủ, Khám phá, and Vé của tôi. Role switcher remains for demonstration.

The FastAPI process wraps SQLAlchemy sessions, business functions, and JWT helpers in `src/` and `api/` [9, 7]. If the `web/` directory exists, the app mounts it as static files so one origin can serve both UI and API [1].

Captured screenshots display the pill "Chế độ: Độc lập" and six mock events. The walkthrough therefore exercises the local fallback, not a live PostgreSQL session.

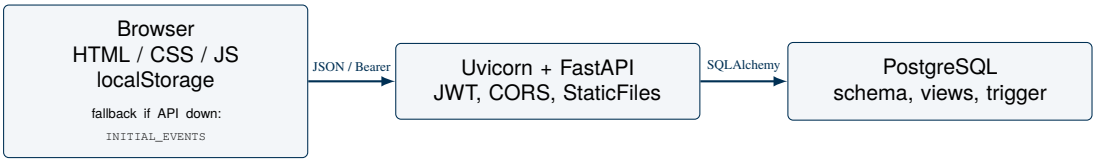


Figure 5. Runtime architecture. The UI always runs. When Uvicorn is absent, the catalogue falls back to `INITIAL_EVENTS` in `app.js`. Screenshots in this report used that independent mode.

2.4. Database design

The schema in `sql/schema.sql` is normalised around events, attendance, finance, and RBAC [10]. Figure 6 is the project ER diagram.

Core entities include events, venues, organisers, categories, and guests. A junction table links events to categories. Attendance and money sit in `registrations` and `event_finance`. Access control uses `users`, `roles`, `permissions`, and `role_permissions`. A user may link to a guest row.

A before-insert trigger on `registrations` raises an exception when the venue is full. Two SQL views pre-aggregate occupancy and revenue. `events.base_price` stores Vietnamese đồng (for example `250000` means

250,000 VND).

2.5. API design

Table 2 lists the FastAPI routes in `api/main.py`. Collection paths use plural nouns; the HTTP verb carries the action [2, 7]. Pydantic models validate JSON bodies. Role guards for Admin and for Organizer-or-Admin wrap `get_current_user`.

Method	Endpoint	Description
POST	`/api/auth/login`	Verify bcrypt hash; return JWT and user payload
POST	`/api/auth/register`	Create guest + user (`role_id=3`); 201 or 409
GET	`/api/events`	List events; optional query filters in Python
GET	`/api/events/{id}`	Event detail; 404 if missing
POST	`/api/events`	Create event (Organizer or Admin)
PUT	`/api/events/{id}`	Replace event fields (Organizer or Admin)
PATCH	`/api/events/{id}/cancel`	Set status `Cancelled` (Admin)
GET	`/api/categories`	Category lookup
GET	`/api/my-bookings`	Registrations for the token's `guest_id`
POST	`/api/bookings`	Register current guest; 201 + `registration_id`
POST	`/api/checkin`	Set `attendance_status` (Organizer or Admin)
GET	`/api/finance/summary`	Rows from `vw_finance_summary`
GET	`/api/attendance/summary`	Rows from `vw_event_attendance_summary`
PUT	`/api/finance/{event_id}`	Update planned budget and actual cost
GET	`/api/venues`	Venue dropdown
GET	`/api/organizers`	Organiser dropdown

Table 2. Published REST endpoints. Paths are taken from `api/main.py`. Check-in on the API requires Organizer or Admin (see Section 4.2).

The next section shows how these contracts appear in code and on the running pages.

3. Implementation

Implementation maps the design onto a static front end and a FastAPI process [7, 4]. Only short excerpts appear here; the full tree is on GitHub (cover page).

3.1. Project structure

```
web_design/
|-- web/                # HTML, CSS, JS storefront
|   |-- landing.html
|   |-- index.html      # explore, bookings, admin SPA
|   |-- events.html
|   |-- event-detail.html
|   |-- checkout.html
|   |-- profile.html
|   |-- terms.html
|   |-- css/  js/
|-- api/                # FastAPI
|   |-- main.py / auth_utils.py / database.py
|-- src/                # SQLAlchemy access layer
|   |-- connection.py / auth.py / crud.py
|-- sql/schema.sql
|-- docs/ERD.png
```

```
|-- requirements.txt
`-- README.md
```

The repository root also contains a leftover TechStore sketch named `index.html`; it is not part of EventHub. `sql/seed.sql` is listed in `.gitignore` and is not in the submitted tree.

3.2. User interface

Figures 7–12 document the captured session of 11 November 2026. The session opened `web/index.html` as a local file URL in independent mode (six mock events).

KPI numbers on the dashboard (for example revenue 4,112,700,000 VND and 11,257 tickets in Figure 11) are computed in the browser from `state.events` as $\sum$ registered $\times$ price. They are not live SQL view output. Landing marketing counters in Figure 15 rendered as $0+$, $0\text{K}+$, 0%, $0+$.

3.3. Server-side logic

Listing 1 issues an HS256 token with an `exp` claim [3]. The default secret is a development string read from `JWT_SECRET_KEY` if present. Expiry is eight hours.

Listing 1. JWT issuance in `api/auth_utils.py`.

```python
DEV_SECRET = "traveloka-event-secret-key-change-in-production"
SECRET_KEY = os.getenv("JWT_SECRET_KEY", DEV_SECRET)
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 8

def create_access_token(data: dict, expires_delta=None) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (
        expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

Listing 2 verifies the password with bcrypt and collects permission names through `role_permissions` [6]. The query inner-joins permissions; a role with no granted permissions yields no rows and login fails.

Listing 2. Password check in `src/auth.py`.

```python
def authenticate_user(session, username, password):
    query = text("""
        SELECT u.user_id, u.password_hash, r.role_name,
               p.permission_name, u.guest_id, g.guest_name, g.guest_email
        FROM users u
        JOIN roles r ON u.role_id = r.role_id
        JOIN role_permissions rp ON r.role_id = rp.role_id
        JOIN permissions p ON rp.permission_id = p.permission_id
        LEFT JOIN guests g ON u.guest_id = g.guest_id
        WHERE u.username = :username
    """)
    result = session.execute(query, {"username": username}).fetchall()
    if result and verify_password(password, result[0][1]):
        perms = [row[3] for row in result]
        return {"user_id": result[0][0], "role": result[0][2],
                "permissions": perms}
    return None
```

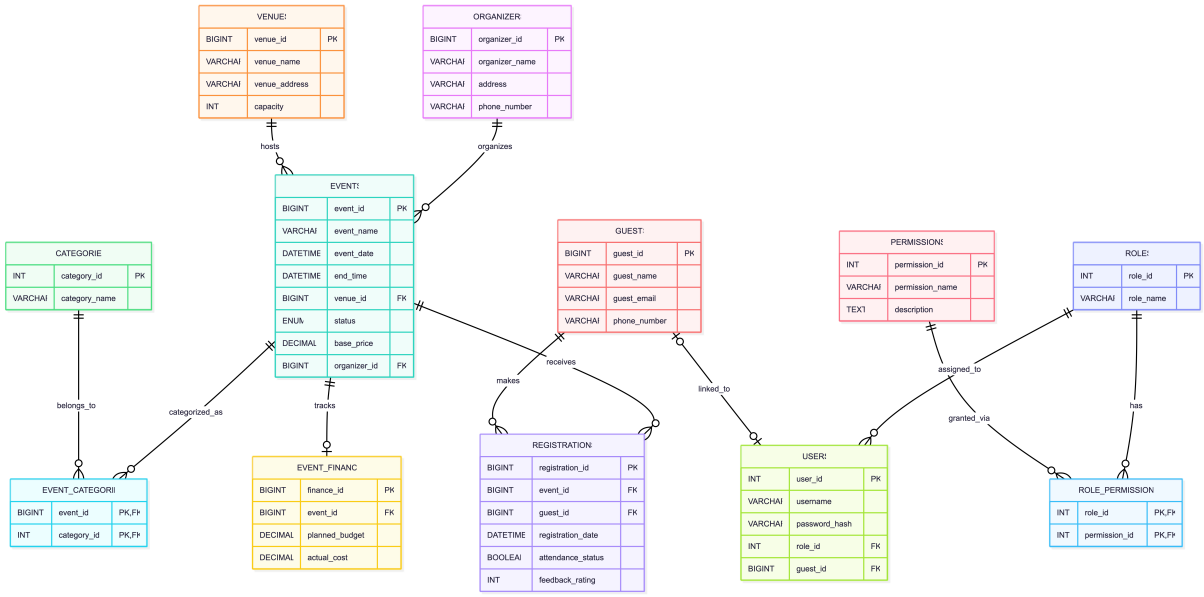


Figure 6. Entity–relationship diagram. Events sit between venues and organisers; registrations connect guests to events; users bind a role and an optional guest profile. Source: `docs/ERD.png`. Table definitions are in `sql/schema.sql`.

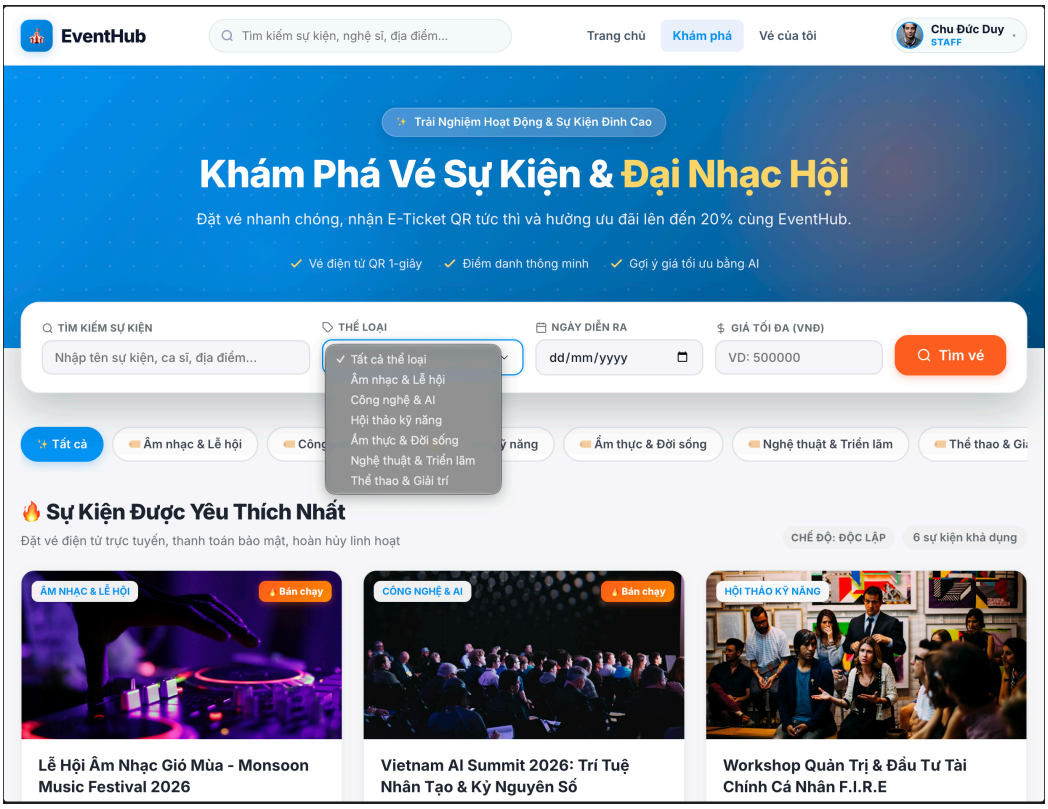


Figure 7. Catalogue filters. Keyword, category list, date picker, and maximum price (VND) on the explore hero. Staff session.

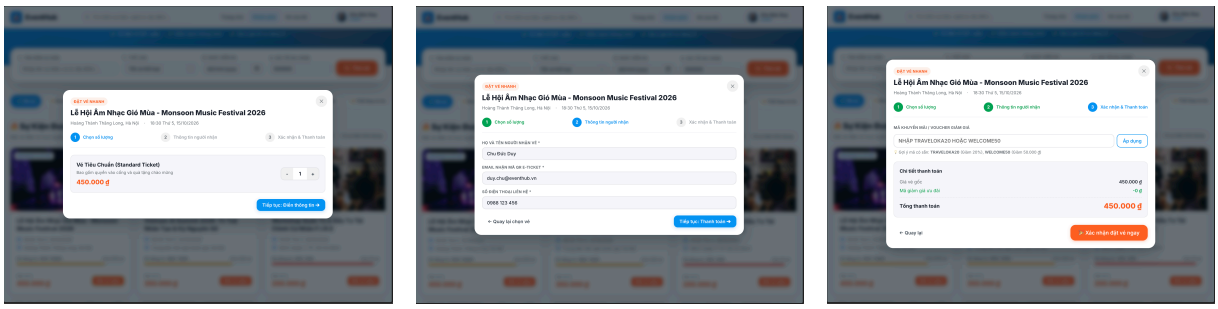


Figure 8. Three-step booking wizard. (1) Standard ticket at 450,000 VND for Monsoon Music Festival 2026. (2) Attendee Chu Đức Duy. (3) Voucher field and payable total before discount.

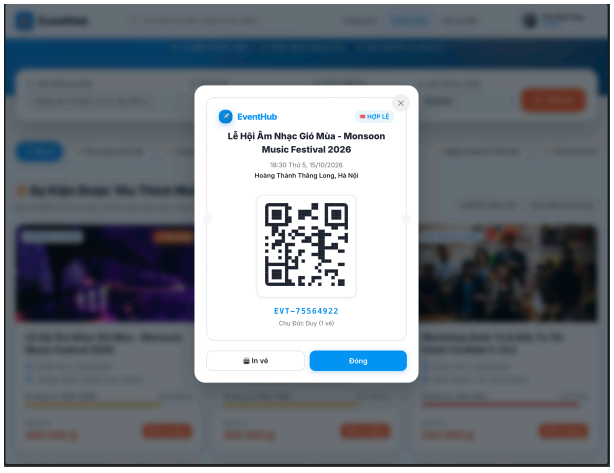


Figure 9. E-Ticket. Code `EVT-75564922` for one Standard ticket, status Hợp lệ. The matrix is an SVG pattern derived from a string hash, not a standards-compliant QR encoder.

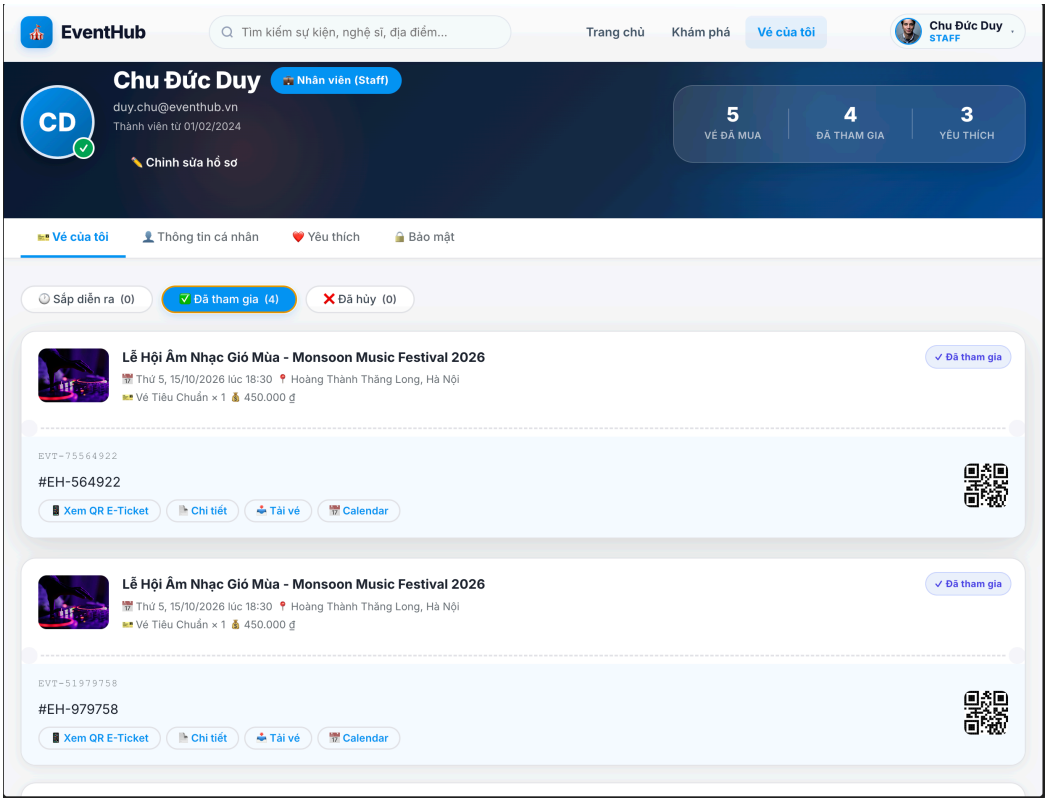


Figure 10. Profile — tickets. Staff account Chu Đức Duy. Counters show 5 purchased, 4 attended, 3 liked. Tab “Đã tham gia (4)” lists issued codes including `EVT-75564922`.

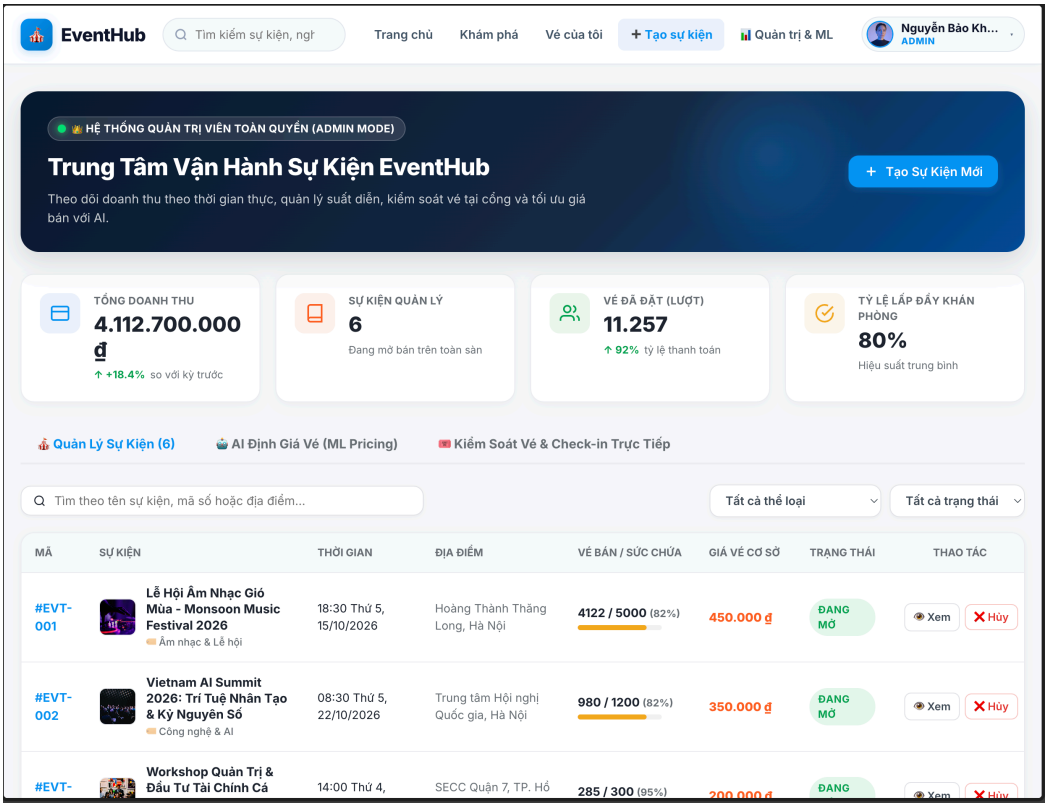


Figure 11. Admin operations console. Four KPIs (revenue, managed events, tickets, occupancy), event table with occupancy bars, and actions Xem / Hủy. Banner: Admin Mode.

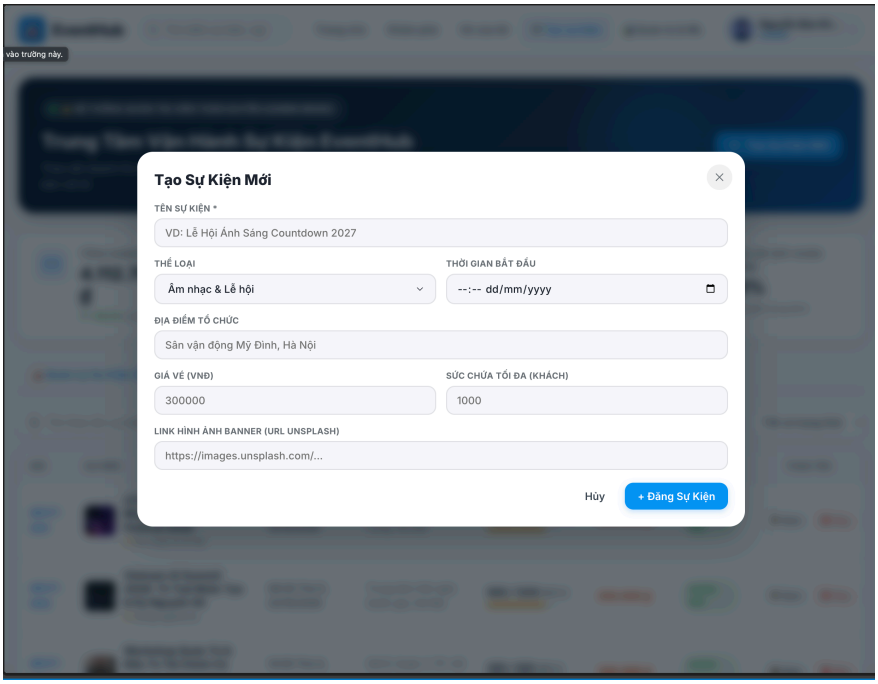


Figure 12. Create-event modal. Name, category, start time, venue, price, capacity, and Unsplash banner URL. Shown to Admin and Organizer only.

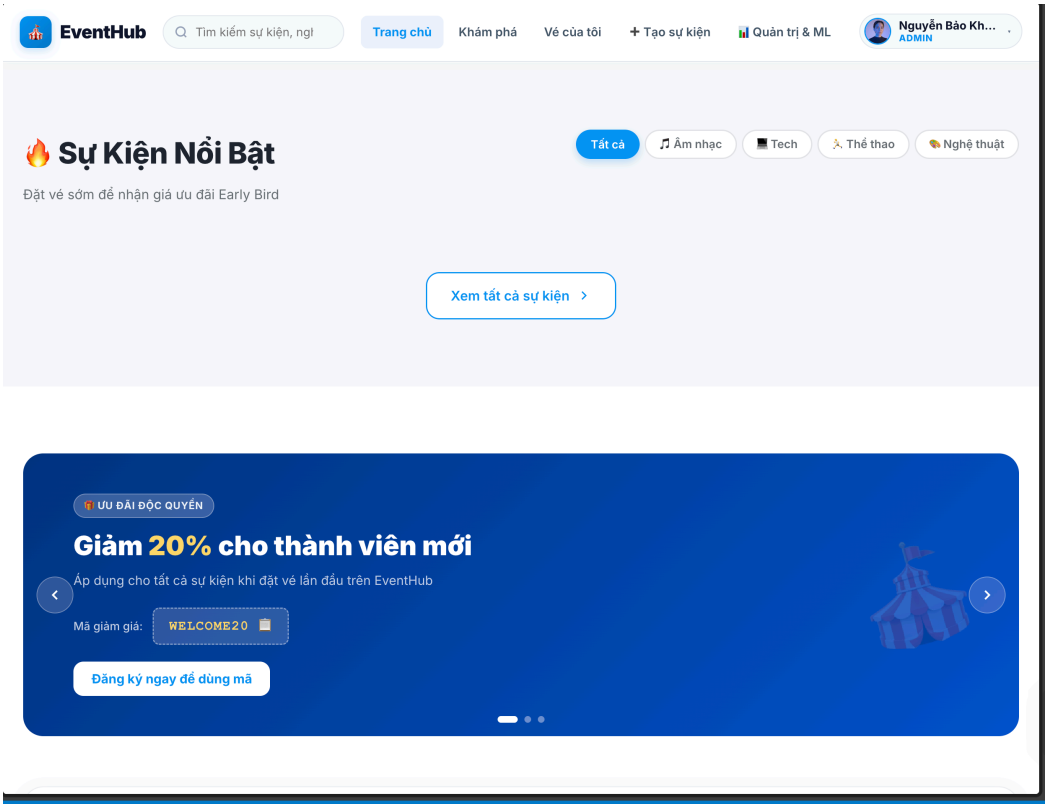


Figure 13. Landing promotion. New-member 20% banner with code `WELCOME20`. The booking wizard instead recognises `TRAVELOKA20` and `WELCOME50`; the two surfaces are not the same code list.

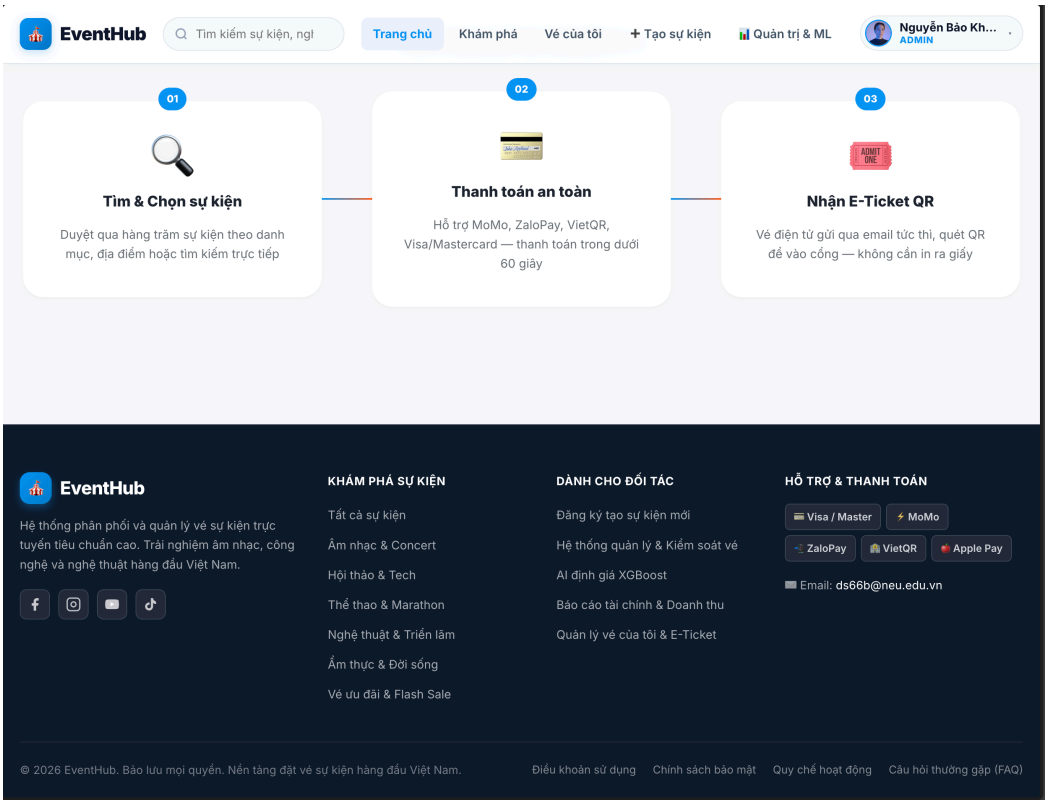


Figure 14. Landing footer. Category links, partner links, payment badges (display only), and contact `ds66b@neu.edu.vn`.

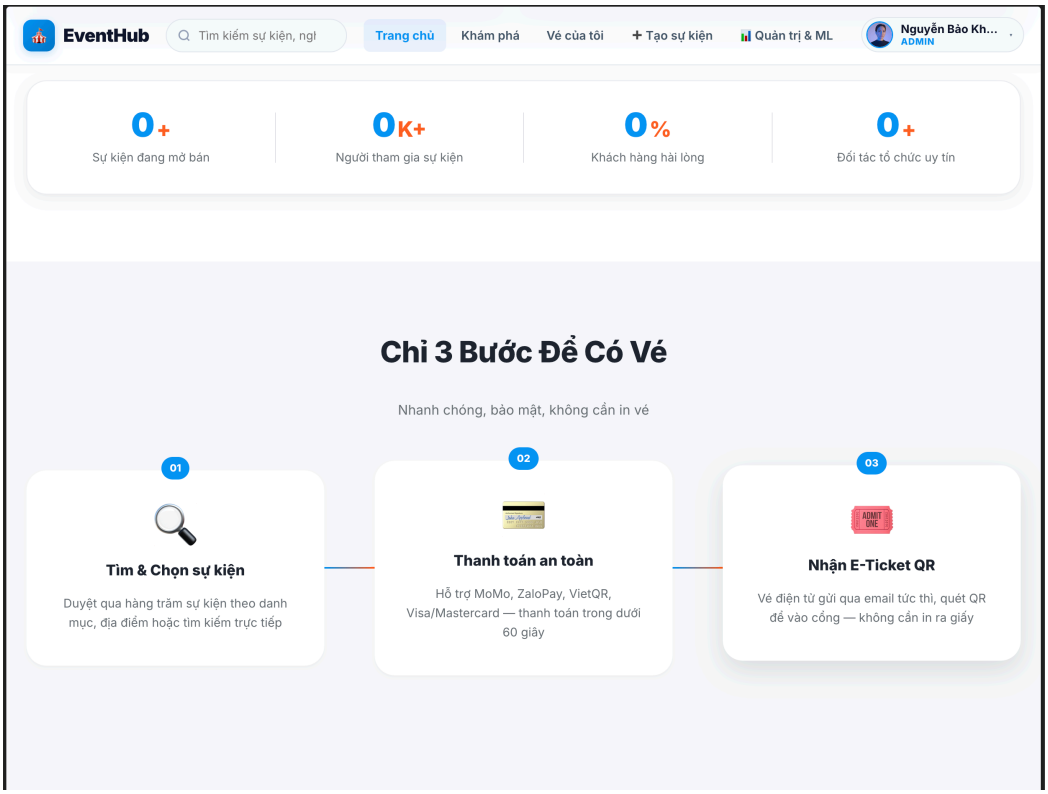


Figure 15. How-to strip and marketing counters. Three steps (search, pay, receive QR). The numeric counters were zero in the captured session.

Listing 3 refuses a duplicate pair (event, guest) and a full venue before insert. The SQL trigger provides a second barrier [10].

Listing 3. Capacity guard in `src/crud.py`.

```
if result[0] > 0:
    return False, "This guest has already registered for this event!"
# venue_capacity vs current_registrations
if current_registrations >= venue_capacity:
    return False, "This event has reached its maximum capacity..."
session.execute(text("""
    INSERT INTO registrations
      (event_id, guest_id, registration_date, attendance_status, feedback_rating)
    VALUES (:eid, :gid, NOW(), false, 0)
"""), {"eid": event_id, "gid": guest_id})
```

The pricing pane is not a fitted model. Equation (1) restates the pricing heuristic in `app.js`: a category multiplier, a capacity multiplier, then rounding to 10,000 VND.

$$P_{\text{std}} = 10000 \cdot \text{round}\left(\frac{P_{\text{base}} \cdot m_{\text{cat}} \cdot m_{\text{cap}}}{10000}\right) \tag{1}$$

Early-bird and VIP tiers are $0.8\,P_{\text{std}}$ and $1.8\,P_{\text{std}}$ after the same rounding. Estimated revenue uses occupancy 0.85 times capacity times P_{std}. The UI label “XGBoost” / “$R^2 = 0.94$” is presentation text; `requirements.txt` does not include scikit-learn, XGBoost, or NumPy.

3.4. Source control and teamwork

The remote is https://github.com/baokhanh1410/web_design (cover page). The group uses a shared repository with HTML/JS on the front end and Python modules on the back end. A detailed branch-and-pull-request log is not included in this tree; [TO BE CONFIRMED: branch names and review practice].

3.5. Deployment

No production host is recorded. Local run: create `.venv`, install `requirements.txt`, fill `.env` (`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`), load `sql/schema.sql`, then

```
uvicorn api.main:app --reload --port 8000
```

Swagger UI is at `http://localhost:8000/docs`. The UI may also be opened as files or via `python3 -m http.server 3000 --directory web`. The captured screenshots used the file protocol, which explains the independent-mode badge.

4. Results and Conclusion

This section reports manual checks against the captured interface, compares them with the objectives in Section 1, and lists residual defects. No Lighthouse run and no pytest suite exist in the repository, so scores of that kind are not invented here.

4.1. Testing

Table 3 records checks performed on 11 November 2026 in independent (mock) mode. The expected result is taken from the specification in Section 2; the observed result is the screenshot set in `figures/`.

ID	Feature	Expected result	Result
TC01	Explore as Admin	Admin links and role dropdown visible	Pass
TC02	Explore as Staff	No create/admin header links	Pass
TC03	Explore as Guest	Reduced navigation; Guest chip	Pass
TC04	Category filter UI	Six categories plus “Tất cả”	Pass
TC05	Date picker	Native calendar, November 2026	Pass
TC06	Booking step 1	Quantity stepper, 450,000 VND	Pass
TC07	Booking step 2	Prefill name and email for Staff	Pass
TC08	Booking step 3	Subtotal 450,000 VND, voucher field	Pass
TC09	E-Ticket after confirm	Code `EVT-75564922`, status Hợp lệ	Pass
TC10	Profile tickets	Ticket listed under “Đã tham gia”	Pass
TC11	Admin dashboard	Four KPIs, event table, cancel action	Pass
TC12	Create-event modal	Required name, venue, price, capacity	Pass
TC13	Organizer dashboard	Organizer Mode banner, same table	Pass
TC14	Landing promo vs wizard	Same voucher catalogue	Fail
TC15	Landing numeric counters	Non-zero marketing statistics	Fail
TC16	Live API badge	“Live API Kết Nối” when server up	Not run

Table 3. Selected manual test cases. TC14–TC15 failed on the captured pages. TC16 was not executed because the session used a local file URL without Uvicorn.

TC14 fails because the two surfaces advertise different codes (Table 4). TC15 fails because `landing.html` paints each counter as `0` until a scroll observer animates it to `data-target`. The screenshot was taken before that observer ran (Figure 15).

4.2. Achievements and limitations

Relative to Section 1, the storefront, role-aware chrome, booking wizard, E-Ticket card, profile, and admin console are implemented and visible in the screenshots. The FastAPI module, SQL schema, views, capacity trigger, bcrypt hashing, and JWT helpers are present in source and match the course stack [7, 10, 3].

Known limitations, all observed in code or in the captured UI:

- Independent mode is what the screenshots show. Booking and check-in in that mode write `localStorage`, not `POST /api/bookings`.
- CORS allows every origin and also enables credentials. Browsers reject that pair, and course material flags it as unsafe [4, 5].
- JWT signing falls back to a hard-coded development secret; expiry is eight hours, so a stolen token remains valid until then [3].
- `authenticate_user` inner-joins `role_permissions`. A role with zero permissions cannot log in through the API.
- The README lists scikit-learn, NumPy, and XGBoost. `requirements.txt` does not. Equation (1) is the implemented estimator.
- Staff check-in is a known defect, not two specs: FR7 and the mock dashboard expose a check-in table to Staff, but `POST /api/checkin` is wrapped in `require_organizer_or_admin`. A Staff token would receive 403. Future work is to add a Staff role guard on that route.
- Voucher catalogues disagree across surfaces (Table 4).
- `sql/seed.sql` is gitignored. Reproducing the 60-event sample requires files that are not in the tree.
- Root `index.html` is an unfinished TechStore page and is unrelated to EventHub.

Surface	Codes accepted
Landing promo banner (Figure 13)	`WELCOME20`
Booking wizard `applyVoucher()`	`TRAVELOKA20` (20%), `WELCOME50` (50,000 VND)

Table 4. Voucher mismatch (TC14). The landing copy and the wizard do not share a catalogue.

4.3. Lessons learned and future work

Building the catalogue and wizard first made the HTTP contract concrete: JSON is the front-end/back-end agreement [2, 7]. The dual-mode design (mock versus API) kept the demo runnable without PostgreSQL, but it also hid integration failures such as TC14 and TC16.

Future work, in priority order:

- Align voucher codes (Table 4); run the landing stat animation on load, not only on scroll.
- Replace `allow_origins=["*"]` with explicit origins; load `JWT_SECRET_KEY` with no default.
- Add pytest + TestClient against an in-memory database, including 401/403/409/422 paths.
- Authorise Staff on `POST /api/checkin` so the gate UI matches the API.
- Swap the SVG hash pattern for a standards-based QR library if gate scanners must read the pass.

- Remove the TechStore root page; publish `seed.sql` or a documented fixture.

References

[1] Encode OSS. Starlette documentation, 2026. Accessed: 2026-11-11.

[2] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

[3] M. Jones, J. Bradley, and N. Sakimura. JSON web token (JWT). RFC 7519, IETF, 2015.

[4] Mozilla. MDN web docs, 2026. Accessed: 2026-11-11.

[5] OWASP Foundation. OWASP top ten, 2021. Accessed: 2026-11-11.

[6] Niels Provos and David Mazières. A future-adaptable password scheme, 1999. USENIX Annual Technical Conference.

[7] Sebastián Ramírez. FastAPI documentation, 2026. Accessed: 2026-11-11.

[8] Jennifer Niederst Robbins. *Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics*. O'Reilly Media, 5 edition, 2018.

[9] SQLAlchemy Authors. SQLAlchemy 2.0 documentation, 2026. Accessed: 2026-11-11.

[10] The PostgreSQL Global Development Group. PostgreSQL documentation, 2026. Accessed: 2026-11-11.

A. Appendix

A.1. Setup and run instructions

```
git clone https://github.com/baokhanh1410/web_design.git
cd web_design
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Create a `.env` file in the project root:

```
DB_HOST=127.0.0.1
DB_PORT=5432
DB_USER=your_postgres_user
DB_PASSWORD=your_postgres_password
DB_NAME=event_management
```

Load the schema (seed data is not in the repository; `sql/seed.sql` is gitignored):

```
psql -U your_postgres_user -d event_management -f sql/schema.sql
uvicorn api.main:app --reload --port 8000
```

Open `http://localhost:8000/docs` for Swagger UI, or open `web/index.html` in a browser for the independent-mode storefront.

A.2. Demo accounts

Table 5 copies the credentials published in the project README. One-click role buttons on the storefront impersonate these profiles in mock mode without calling the API.

Username	Password	Role	Note
`admin`	`admin123`	Admin	Nguyễn Bảo Khánh; full dashboard
`organizer01`	`org123`	Organizer	Trần Quang Huy; create and manage events
`staff01`	`staff123`	Staff	Chu Đức Duy; gate UI in mock mode
`guest01`	`guest123`	Guest	Nguyễn Văn Khang; browse and book

Table 5. Demo accounts. Source: project README. Mock-mode role buttons do not call `/api/auth/login`.

A.3. Use of AI tools

This LaTeX report was prepared from the EventHub source tree and from screenshots dated 11 November 2026. Any additional assistants used for coding should be named here before submission.